LAPORAN TUGAS KELOMPOK

Mata Kuliah Sistem Operasi (USA-WP2260110)

Implementasi Komunikasi Antarproses (IPC)

Kode dan judul tugas	K-1 Implementasi Komunikasi Antarproses (IPC)
Kelas / tema	Kelas B — Perangkat Mobile dan Aplikasi Android
Nama kelompok	Kelompok 6
Nama host VM	Tidak memakai VM, program dijalankan langsung di Windows 11 pada laptop anggota
Minggu / tanggal kumpul	Minggu ke-4 / 14 Oktober 2026
Kepingan puzzle	Kepingan 4 → Level 2 Puzzle UTS
Dosen pengampu	Muhammad Agreindra Helmiawan, S.Kom., M.T

Anggota kelompok

No	Nama	NIM	Peran
1	Nazwa Salsabilla R	250660121077	Leader
2	Satrio Pratama	250660121095	Anggota 1
3	Rizky Imam Andieka	250660121054	Anggota 2

Program Studi S1 Informatika
Fakultas Teknologi Informasi · Universitas Sebelas April
2026

Pernyataan Keaslian

Kami yang bertanda tangan di bawah ini menyatakan bahwa laporan ini merupakan hasil kerja kelompok kami sendiri. Seluruh sumber yang dikutip telah dicantumkan, penggunaan AI generatif telah diungkapkan pada Lampiran, dan setiap anggota berkontribusi sesuai lembar kontribusi.

Nama / NIM	Tanda tangan	Tanggal
Nazwa Salsabilla R 250660121077		14 Oktober 2026
Satrio Pratama 250660121095		14 Oktober 2026
Rizky Imam Andieka 250660121054		14 Oktober 2026

Daftar Isi

Pernyataan Keaslian 2
1. Pendahuluan 4
1.1 Latar belakang kasus 4
1.2 Tujuan tugas 4
1.3 Ruang lingkup 4
2. Pembagian Peran dan Rencana Kerja 4
3. Dasar Teori Singkat 5
4. Lingkungan Praktik 5
5. Langkah Pengerjaan 6
5.1 Langkah 1: Menyiapkan folder kerja dan berkas program 6
5.2 Langkah 2: Merancang dan menjalankan alur normal (pipe dan socket) 6
5.3 Langkah 3: Menguji server mati dan antrian offline 8
5.4 Langkah 4: Menguji pengiriman data besar (1999 catatan) 10
5.5 Langkah 5: Membandingkan server lama dan server baru 11
5.6 Langkah 6: Kendala yang ditemui dan penanganannya 12
6. Hasil dan Analisis 12
6.1 Hasil 12
6.2 Pengujian 13
6.3 Analisis 14
6.4 Rekomendasi 15
7. Kesimpulan 16
8. Kepingan Puzzle 16
Daftar Pustaka 16
Lampiran A. Kode Program dan Log 17
Lampiran B. Lembar Kontribusi Anggota 20
Lampiran C. Log Kerja Kelompok 21
Lampiran D. Pernyataan Penggunaan AI Generatif 21
Lampiran E. Daftar Periksa Sebelum Mengumpulkan 22
Swa-nilai berdasarkan kriteria lembar tugas 22

1. Pendahuluan

1.1 Latar belakang kasus

Sebuah aplikasi Android tersusun dari beberapa komponen, misalnya antarmuka pengguna, layanan latar belakang, dan penyimpanan lokal, yang dapat berjalan sebagai proses berbeda. Setiap proses memiliki ruang alamat sendiri sehingga tidak bisa membaca variabel proses lain, data hanya dapat berpindah melalui mekanisme komunikasi antarproses (inter-process communication, IPC) yang disediakan sistem operasi (Silberschatz dkk., 2018). Pada Android mekanisme ini berupa Binder dan Intent, dan lembar tugas meminta analoginya disimulasikan dengan pipe dan socket.

Kasus kelas adalah sinkronisasi data offline: aplikasi mencatat data ketika perangkat tidak terhubung, lalu mengirimkannya ke server begitu koneksi tersedia. Masalah sistem operasi yang muncul adalah bagaimana data berpindah antarproses dengan benar, yaitu tidak hilang saat server mati, tidak terpotong saat jumlahnya besar, dan tidak membuat dua proses saling menunggu tanpa ujung. Karena itu implementasi IPC beserta pengujiannya penting untuk kasus ini.

1.2 Tujuan tugas

Tugas ini berada pada Sub-CPMK-4, CPMK061, CPMK063, CPMK081, CPMK102, dan CPMK112 (lembar tugas K-1). Dengan kalimat kelompok, tujuan tugas adalah:

- Menjelaskan mekanisme IPC yang dipakai, yaitu message passing melalui pipe antarproses dan socket TCP antara klien dan server.
- Mengimplementasikan simulasi sinkronisasi data offline ke server, menguji program pada beberapa skenario, dan mendokumentasikan hasilnya.
- Menganalisis kelebihan dan keterbatasan mekanisme tersebut untuk kasus kelas Perangkat Mobile dan Aplikasi Android.

1.3 Ruang lingkup

Sistem yang diamati adalah dua program Python, client.py dan server.py, yang berjalan di satu laptop dan berkomunikasi melalui alamat lokal 127.0.0.1 port 5000. Data berupa catatan sintetis berformat JSON dengan tiga isian (id, data, waktu); percobaan memakai 5 catatan untuk pengujian fungsi dan 1999 catatan untuk pengujian volume. Tugas ini tidak mencakup perangkat Android atau emulator, shared memory, enkripsi dan autentikasi, serta banyak klien yang terhubung bersamaan.

2. Pembagian Peran dan Rencana Kerja

Peran dan tanggung jawab dicatat sebelum pengerjaan dan diperbarui jika ada perubahan.

Anggota	Peran	Tanggung jawab	Tenggat
Nazwa Salsabilla R	Penyusun laporan	Koordinasi kelompok, menyusun laporan dan analisis (Bab 1, 3, 6, 7)	14 Oktober 2026
Satrio Pratama	Pengembang klien dan penguji	Program client.py (pipe dan socket), pengujian beban dan offline	14 Oktober 2026
Rizky Imam Andieka	Pengembang server dan dokumentasi	Program server.py, perbaikan server, tangkapan layar dan lampiran	14 Oktober 2026

3. Dasar Teori Singkat

IPC adalah cara proses-proses yang terisolasi bertukar data. Ada dua model dasar: shared memory, ketika proses berbagi satu wilayah memori, dan message passing, ketika data dikirim sebagai pesan melalui kernel (Silberschatz dkk., 2018). Message passing lebih mudah dipakai karena sinkronisasi ditangani sistem operasi, sedangkan shared memory menuntut pemrogram mengatur akses bersama sendiri. Tugas ini memakai message passing.

Pipe adalah kanal di dalam kernel: satu proses menulis ke satu ujung, proses lain membaca dari ujung yang lain, dan pembaca menunggu (blocking) bila data belum ada (Arpaci-Dusseau & Arpaci-Dusseau, 2018; Kerrisk, 2010). Pada pipe biasa kedua proses umumnya berkerabat karena ujung pipe diwariskan dari proses induk. Di Python, multiprocessing.Pipe() menghasilkan sepasang objek koneksi; pada tugas ini dipakai satu arah, yaitu penyimpan menulis dan pengirim membaca.

Socket adalah titik akhir komunikasi yang diidentifikasi oleh alamat IP dan nomor port. Socket aliran (TCP) menyediakan byte stream yang andal dan berurutan, tetapi tidak menjaga batas antarpesan: satu kali recv() dapat mengembalikan sebagian data saja, sehingga aplikasi harus menentukan sendiri batas pesan dan membaca berulang sampai selesai (Kerrisk, 2010). Prinsip ini menjadi inti analisis pada Bagian 6.3.

4. Lingkungan Praktik

Komponen	Keterangan
Laptop (host)	Dijalankan pada tiga laptop anggota. Nazwa – Windows 11 Home Single Language, 11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz, 16 GB Satrio – Windows 11 pro, ryzen 5 5500u, 16 GB Rizky – Windows 11 pro, ryzen 5 3500U, 16GB
Hypervisor	Tidak berlaku (tidak memakai hypervisor)

Komponen	Keterangan
VM	Tidak berlaku; program dijalankan langsung di Windows tanpa mesin virtual
Perangkat/emulator lain	Tidak berlaku; simulasi dijalankan di laptop tanpa Android Emulator atau ponsel
Alat dan bahasa	Python 3 (modul socket, json, time, multiprocessing; peluncur py), Windows PowerShell dan Command Prompt, VS Code, Notepad
Repositori	https://github.com/NazwaSR0/SO_k-1_kelompok-6/upload/main

5. Langkah Pengerjaan

Langkah dikerjakan di folder ipc_Kelompok6. Server dan klien dijalankan di dua jendela terminal terpisah. Klien mencetak PID tiap proses sebagai bukti bahwa komunikasi terjadi antarproses.

5.1 Langkah 1: Menyiapkan folder kerja dan berkas program

Folder kerja berisi tiga berkas: client.py (klien dengan dua proses), server.py (server versi perbaikan), dan server_Lama.py (server versi awal yang disimpan sebagai pembanding pengujian). Kode lengkap ada pada Lampiran A.

```
PS C:\Users\satrio_\Documents\ipc_Kelompok6> dir
client.py       1926
server.py       1109
server_Lama.py   639
```

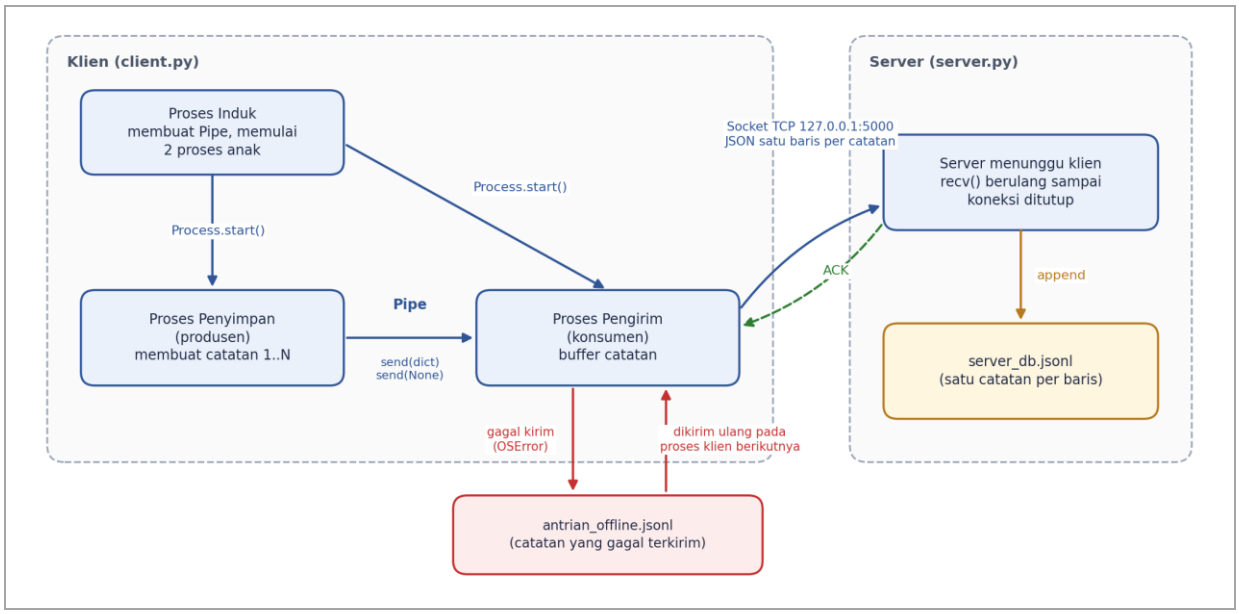


Gambar 5.1 Isi folder ipc_Kelompok6: client.py, server.py, dan server_Lama.py.

5.2 Langkah 2: Merancang dan menjalankan alur normal (pipe dan socket)

Klien terdiri dari proses induk yang membuat Pipe lalu menjalankan dua proses anak. Proses Penyimpan membuat catatan dan mengirimnya ke pipe, kemudian mengirim None sebagai tanda selesai. Proses Pengirim membaca pipe sampai tanda selesai, menampung catatan di buffer, lalu membuka koneksi TCP ke server dan mengirim seluruh catatan sebagai teks JSON,

satu baris per catatan. Server menyimpan tiap catatan ke server_db.jsonl dan membalas ACK. Alur ini digambarkan pada Gambar 5.2.



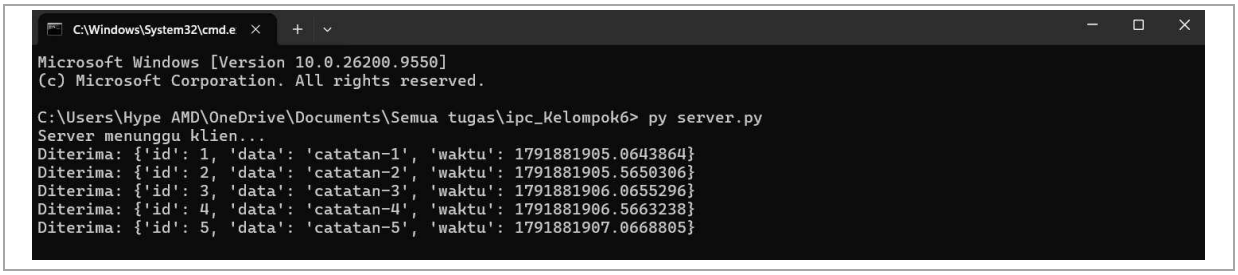
Gambar 5.2 Diagram alur komunikasi: pipe antara proses Penyimpan dan Pengirim, socket TCP ke server, dan jalur antrian offline.

Percobaan pertama memakai server_Lama.py dan menghasilkan catatan id 1 sampai 5 (Gambar 5.3). Setiap percobaan menambah baris di server_db.jsonl (Gambar 5.4). Percobaan dengan versi akhir menampilkan tiga PID berbeda dan balasan ACK (Gambar 5.5), dan hasil serupa diperoleh di laptop anggota lain (Gambar 5.6).

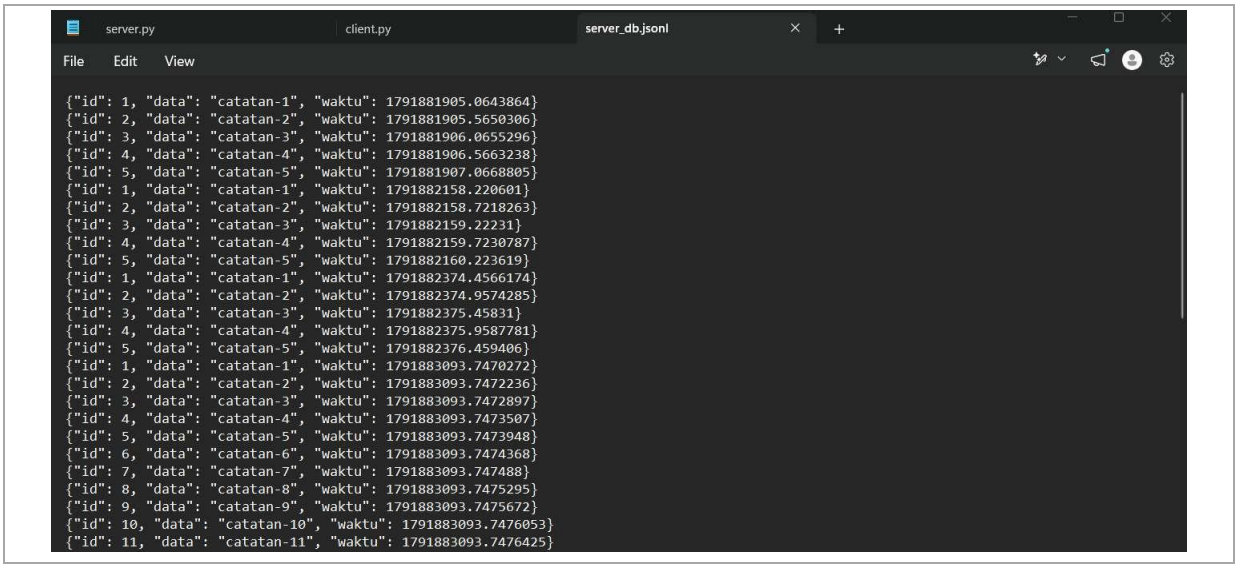
```
# Terminal 1
PS ...\ipc_Kelompok6> py server.py
Server menunggu klien...

# Terminal 2
PS ...\ipc_Kelompok6> py client.py
Induk      pid = 10300
Penyimpan  pid = 7088
Pengirim   pid = 12636
Balasan server: ACK

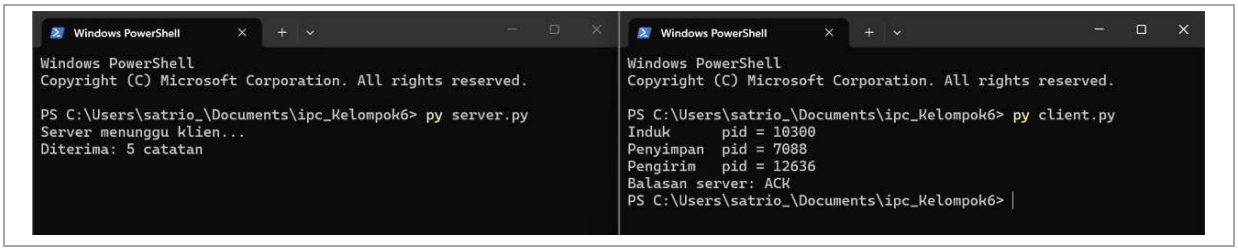
# Terminal 1 setelah klien selesai
Diterima: 5 catatan
```



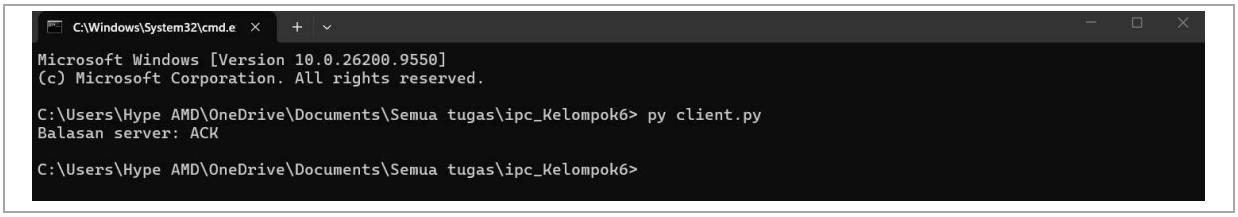
Gambar 5.3 Percobaan pertama dengan server_Lama.py: server menerima catatan id 1 sampai 5.



Gambar 5.4 Isi server_db.jsonl setelah beberapa percobaan: id 1 sampai 5 muncul berulang karena server menambahkan setiap kiriman.



Gambar 5.5 Alur normal dengan versi akhir, tiga PID berbeda (Induk, Penyimpan, Pengirim), server menerima 5 catatan dan klien menerima ACK.



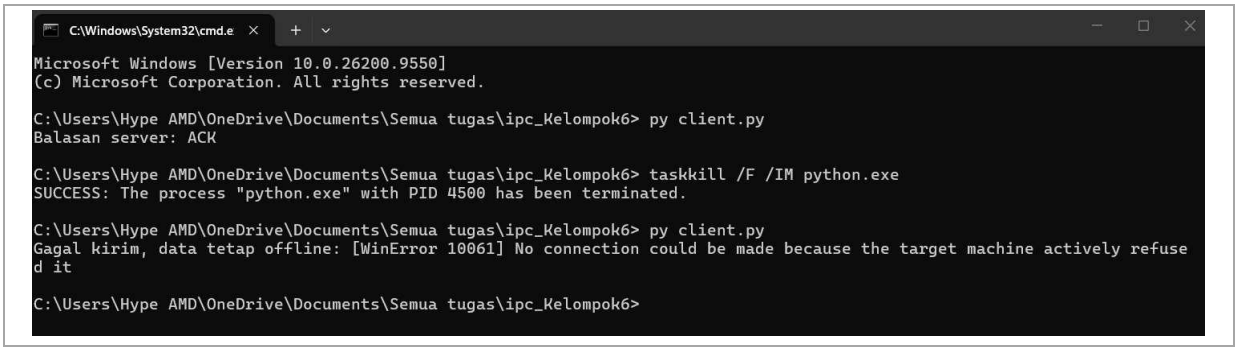
Gambar 5.6 Pengujian ulang pada laptop anggota lain: klien menerima balasan ACK dari server.

5.3 Langkah 3: Menguji server mati dan antrian offline

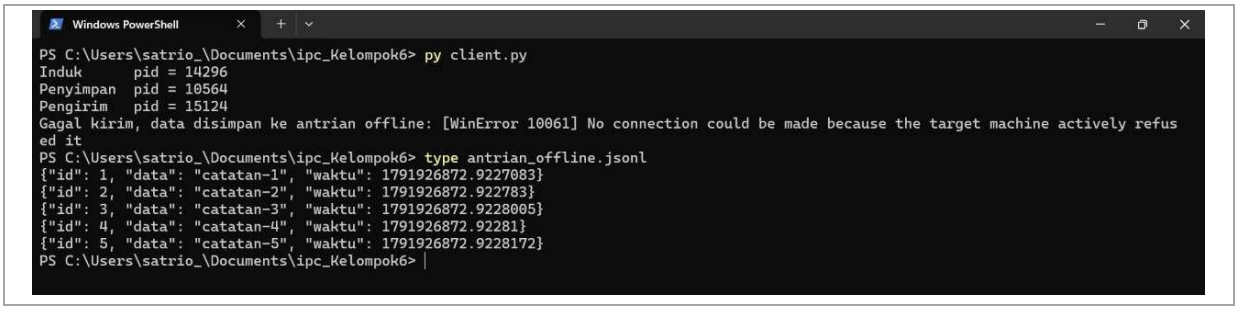
Untuk meniru perangkat yang kehilangan koneksi, server dimatikan lalu klien dijalankan. Koneksi ditolak (WinError 10061, Gambar 5.7) dan pada versi akhir klien menyimpan catatan ke antrian_offline.jsonl (Gambar 5.8). Setelah server dijalankan lagi dan klien dijalankan sekali

lagi, server menerima 10 catatan, yaitu 5 dari antrian dan 5 catatan baru, dan berkas antrian tidak ada lagi di folder (Gambar 5.9).

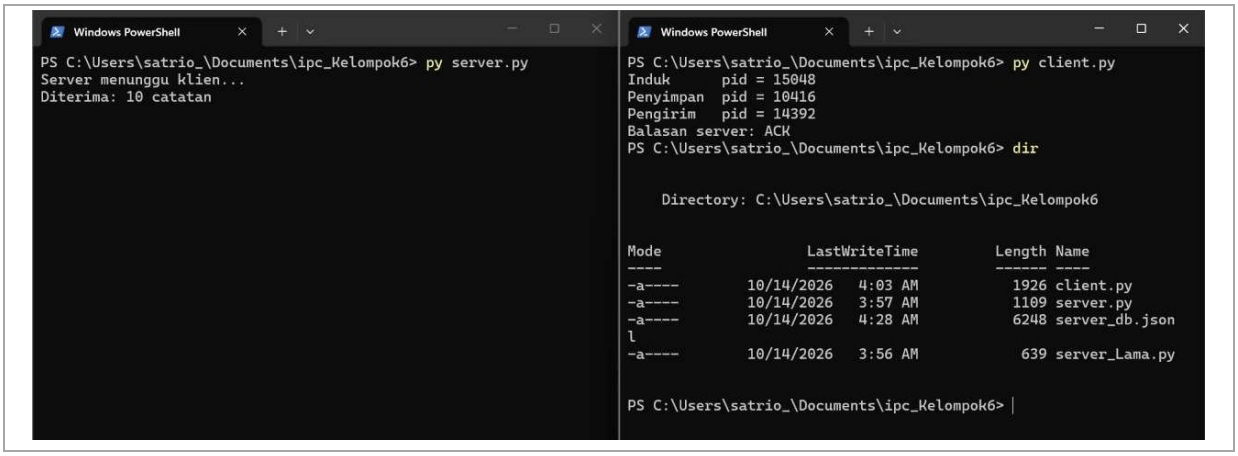
```
PS ...\ipc_Kelompok6> taskkill /F /IM python.exe
SUCCESS: The process "python.exe" with PID 4500 has been terminated.
PS ...\ipc_Kelompok6> py client.py
Gagal kirim, data disimpan ke antrian offline: [WinError 10061] ...
PS ...\ipc_Kelompok6> type antrian_offline.jsonl
{"id": 1, "data": "catatan-1", "waktu": 1791926872.9227083}
... (5 baris, id 1 sampai 5)
# server dijalankan lagi, lalu klien dijalankan lagi
Balasan server: ACK
Diterima: 10 catatan   (tampil di terminal server)
```



Gambar 5.7 Server dimatikan dengan taskkill; klien berikutnya gagal terhubung (WinError 10061, koneksi ditolak).



Gambar 5.8 Versi akhir klien: kegagalan yang sama menyimpan 5 catatan ke antrian_offline.jsonl.

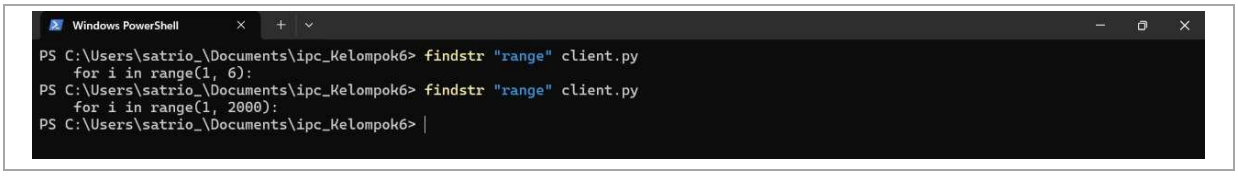


Gambar 5.9 Server dihidupkan kembali: server menerima 10 catatan (5 dari antrian dan 5 baru), dan antrian_offline.jsonl tidak lagi ada pada daftar berkas.

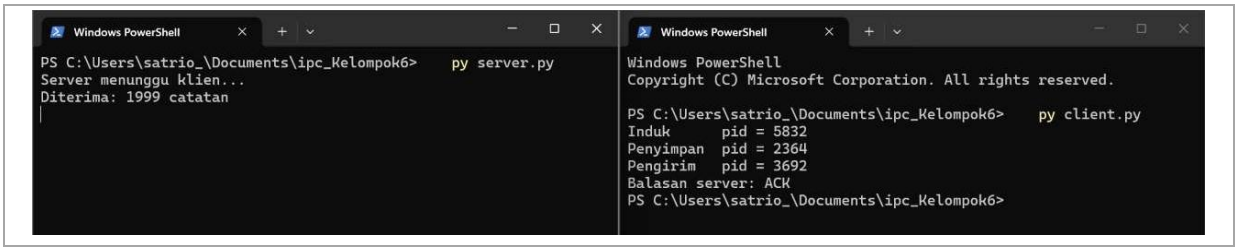
5.4 Langkah 4: Menguji pengiriman data besar (1999 catatan)

Jumlah catatan pada klien diubah dari range(1, 6) menjadi range(1, 2000) sehingga menghasilkan 1999 catatan, sekitar 127 KB dalam format JSON per baris, jauh di atas 4096 byte. Pengujian dijalankan dengan server.py (Gambar 5.10, Gambar 5.11), lalu isi server_db.jsonl diperiksa (Gambar 5.12).

```
PS ...\ipc_Kelompok6> findstr "range" client.py
    for i in range(1, 2000):
PS ...\ipc_Kelompok6> py client.py
Balasan server: ACK          # terminal server: Diterima: 1999 catatan
PS ...\ipc_Kelompok6> Get-Content server_db.jsonl -Tail 1
{"id": 1999, "data": "catatan-1999", "waktu": 1791931751.6597812}
PS ...\ipc_Kelompok6> (Get-Content server_db.jsonl).Count
1999
```



Gambar 5.10 Pengecekan perintah findstr pada client.py: jumlah catatan diubah dari range(1, 6) menjadi range(1, 2000).



Gambar 5.11 Pengiriman 1999 catatan dengan server.py: server mencatat 1999 catatan dan klien menerima ACK.

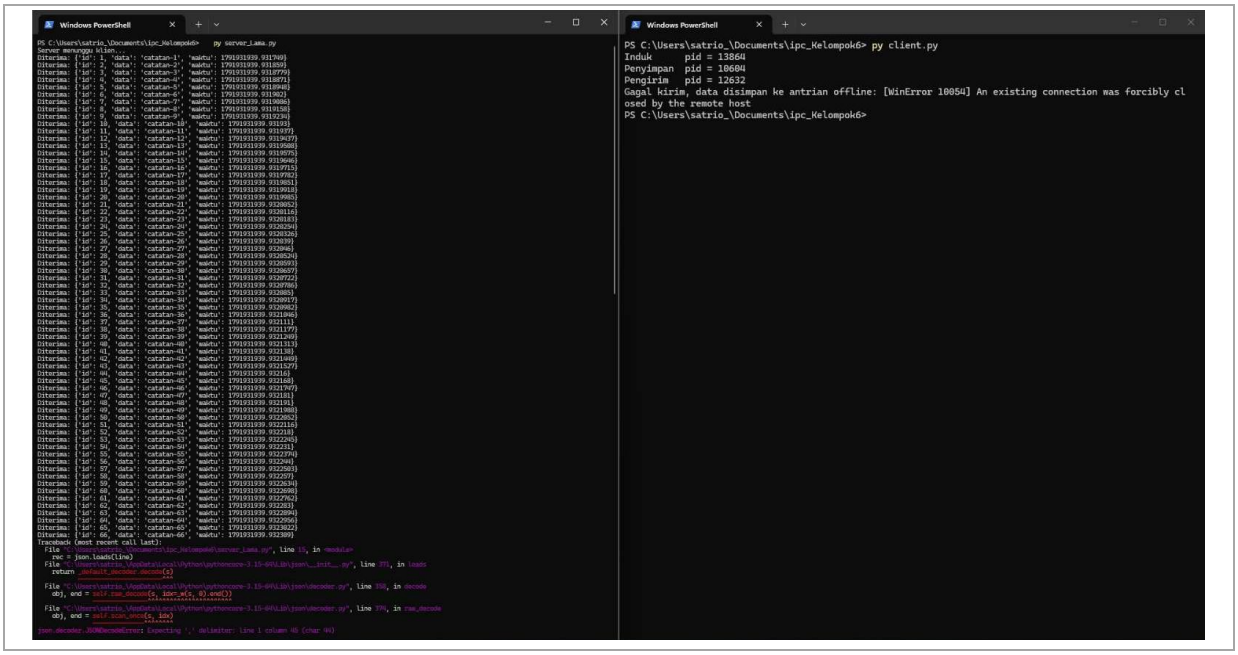


Gambar 5.12 Verifikasi server_db.jsonl: baris terakhir berisi id 1999 dan jumlah baris tepat 1999.

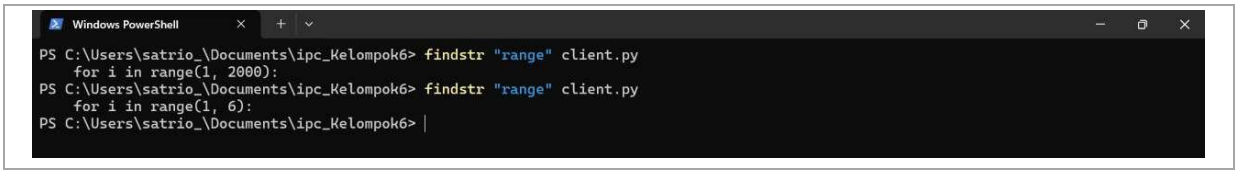
5.5 Langkah 5: Membandingkan server lama dan server baru

Pengujian yang sama dijalankan dengan server_Lama.py, yang membaca data dengan satu kali recv(4096). Server hanya berhasil memproses catatan sampai id 66, lalu berhenti dengan JSONDecodeError karena baris berikutnya terpotong; klien menerima WinError 10054 dan menyimpan data ke antrian (Gambar 5.13). Setelah pengujian selesai, nilai range pada client.py dikembalikan ke (1, 6) (Gambar 5.14).

```
PS ...\ipc_Kelompok6> py server_Lama.py
Diterima: {'id': 66, 'data': 'catatan-66', ...}
json.decoder.JSONDecodeError: Expecting ',' delimiter: line 1 column 45 (char 44)
PS ...\ipc_Kelompok6> py client.py
Gagal kirim, data disimpan ke antrian offline: [WinError 10054] ...
PS ...\ipc_Kelompok6> findstr "range" client.py
    for i in range(1, 6):
```



Gambar 5.13 Pengujian 1999 catatan dengan server_Lama.py: server berhenti setelah catatan id 66 dengan JSONDecodeError, klien menerima WinError 10054 dan menyimpan data ke antrian.



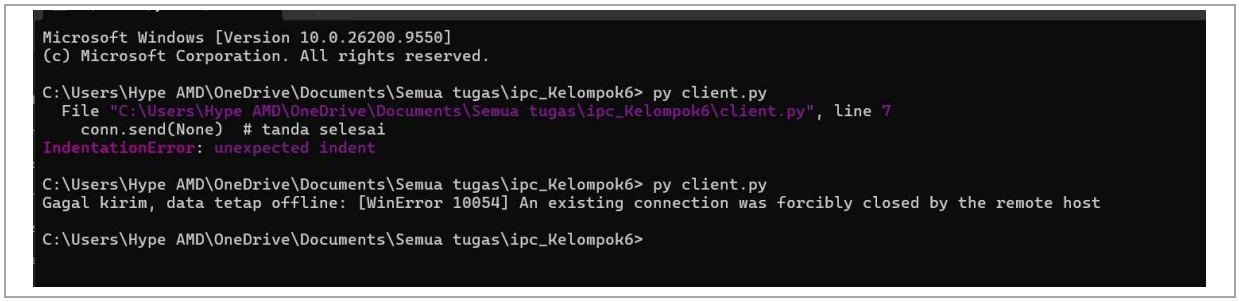
Gambar 5.14 Setelah pengujian volume, nilai range pada client.py dikembalikan dari 2000 menjadi 6.

5.6 Langkah 6: Kendala yang ditemui dan penanganannya

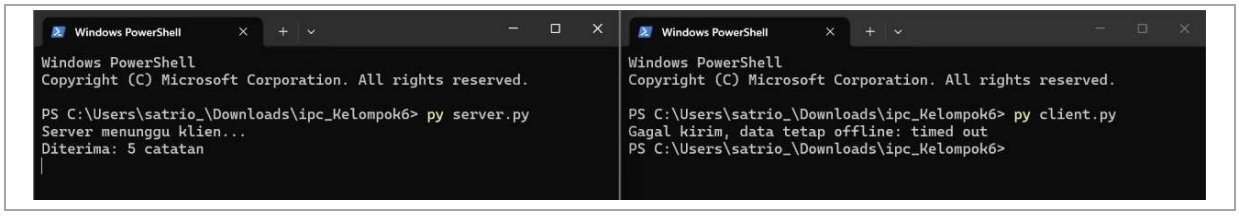
Kendala pertama adalah IndentationError pada baris 7 client.py (conn.send(None)) karena indentasi berlebih. Setelah dirapikan klien berjalan, tetapi koneksi diputus oleh server (WinError 10054), gejala yang sama dengan berhentinya server_Lama.py pada Langkah 5 (Gambar 5.15).

Kendala kedua muncul pada salinan awal program: klien melaporkan timed out walaupun server menampilkan "Diterima: 5 catatan" (Gambar 5.16). Server.py membaca sampai koneksi ditutup, sedangkan klien menunggu ACK tanpa menutup sisi kirimnya, sehingga keduanya saling menunggu sampai batas waktu 5 detik habis; server baru memproses data setelah klien menyerah dan menutup koneksi. Perbaikannya, klien menutup sisi kirim koneksi (shutdown SHUT_WR) setelah mengirim, sehingga server tahu data sudah lengkap dan membalas ACK (Kerrisk, 2010).

```
IndentationError: unexpected indent          # client.py baris 7
Gagal kirim, data tetap offline: timed out   # klien menunggu ACK
```



Gambar 5.15 Kendala 1: IndentationError pada baris 7 client.py, lalu klien berjalan tetapi koneksi diputus server (WinError 10054).



Gambar 5.16 Kendala 2: server menampilkan "Diterima: 5 catatan" tetapi klien melaporkan timed out karena menunggu ACK.

6. Hasil dan Analisis

6.1 Hasil

Tabel berikut merangkum hasil observasi seluruh percobaan; bukti ada pada gambar yang dirujuk.

No	Item yang diamati/dihitung	Hasil	Keterangan
1	Jumlah proses pada klien	3 proses (induk, Penyimpan, Pengirim)	Gambar 5.5

No	Item yang diamati/dihitung	Hasil	Keterangan
		dengan PID berbeda, contoh 10300, 7088, dan 12636	
2	Pengiriman normal 5 catatan	Server: "Diterima: 5 catatan"; klien: "Balasan server: ACK"	Gambar 5.5
3	Pengiriman saat server mati	WinError 10061; 5 catatan masuk ke antrian_offline.jsonl	Gambar 5.7, Gambar 5.8
4	Sinkronisasi antrian setelah server hidup	10 catatan diterima (5 antrian + 5 baru); berkas antrian tidak ada lagi	Gambar 5.9
5	Pengiriman 1999 catatan, server.py	1999 catatan diterima; server_db.jsonl berisi 1999 baris dengan id terakhir 1999	Gambar 5.11, Gambar 5.12
6	Pengiriman 1999 catatan, server_Lama.py	Berhenti setelah id 66 dengan JSONDecodeError; klien menerima WinError 10054	Gambar 5.13

6.2 Pengujian

Pengujian dilakukan pada program IPC dengan enam skenario berikut. Status "Gagal" pada baris 5 dan 6 menunjukkan kegagalan pada versi awal yang kemudian diperbaiki (Bagian 5.5 dan 5.6).

No	Skenario uji	Hasil diharapkan	Hasil aktual	Status
1	Kirim 5 catatan saat server hidup	Server menerima 5 catatan dan membalas ACK	Server: "Diterima: 5 catatan"; klien: ACK	Lulus
2	Kirim catatan saat server mati	Klien tidak berhenti dan	WinError 10061; 5 catatan tersimpan di antrian_offline.jsonl	Lulus

No	Skenario uji	Hasil diharapkan	Hasil aktual	Status
		data tidak hilang		
3	Jalankan server lagi, lalu kirim ulang	Antrian dan catatan baru terkirim	Server: "Diterima: 10 catatan"; antrian terhapus	Lulus
4	Kirim 1999 catatan ke server.py	1999 catatan tersimpan utuh	Server_db.jsonl berisi 1999 baris, id terakhir 1999	Lulus
5	Kirim 1999 catatan ke server_Lama.py	Seluruh catatan diterima	Berhenti pada id 66 (JSONDecodeError), klien menyimpan ke antrian	Gagal (diperbaiki pada server.py)
6	Klien awal menunggu ACK tanpa menutup sisi kirim	Klien menerima ACK	Timed out setelah 5 detik; server baru memproses setelah klien menutup koneksi	Gagal (diperbaiki pada klien akhir)

6.3 Analisis

Analisis berikut menjawab tiga poin pada deskripsi tugas secara berurutan.

Poin (1): Mekanisme IPC yang dipakai

- Klien memakai message passing dalam dua tahap. Pada tahap pertama, proses Penyimpan (produsen) dan Pengirim (konsumen) bertukar catatan melalui pipe. Tiga PID berbeda pada Gambar 5.5 menunjukkan bahwa ketiganya proses terpisah dengan ruang alamat sendiri, sehingga data harus disalin lewat kernel dan tidak bisa dibaca langsung dari variabel proses lain.
- Pengirim memanggil recv() pada pipe dan menunggu jika data belum ada. Sifat blocking ini sudah menyinkronkan produsen dan konsumen tanpa semaphore, dan None dikirim sebagai penanda akhir agar pengirim tahu kapan berhenti membaca.
- Pada tahap kedua, pengirim mengirim data ke server melalui socket TCP. Satu baris JSON per catatan menjadi aturan batas pesan, dan ACK dari server menjadi konfirmasi di tingkat aplikasi. Message passing dipilih dibanding shared memory karena data harus sampai ke proses lain yang berperan sebagai server, sedangkan shared memory hanya bekerja pada satu mesin dan menuntut sinkronisasi manual (Silberschatz dkk., 2018).

Poin (2): Pengujian program dan hasilnya

- Skenario normal, server mati, dan sinkronisasi antrian berjalan sesuai harapan (Tabel 6.2 no. 1 sampai 3), dan pengiriman 1999 catatan dengan server.py menghasilkan 1999 baris utuh (no. 4).
- Dua kegagalan ditemukan dan diperbaiki. Pertama, server_Lama.py membaca dengan satu kali recv(4096). TCP hanya menjamin urutan byte, bukan batas pesan, sehingga data yang lebih besar dari sekitar 4 KB (kira-kira 66 catatan dengan panjang sekitar 62 byte) terpotong di tengah baris dan json.loads gagal. Server baru membaca berulang sampai koneksi ditutup, memecah per baris, dan melewati baris rusak. Kedua, klien awal menunggu ACK sebelum menutup sisi kirim, sementara server menunggu penutupan koneksi, sehingga keduanya saling menunggu sampai timeout (Kerrisk, 2010).
- Gambar 5.4 memperlihatkan id 1 sampai 5 muncul berulang di server_db.jsonl: server menambahkan setiap catatan yang datang tanpa memeriksa duplikat.

Poin (3): Kelebihan dan keterbatasan untuk kasus kelas

- Kelebihan: proses terisolasi dan sinkronisasi produsen-konsumen sederhana karena pipe bersifat blocking.
- Kelebihan: pencatatan data terpisah dari pengiriman, sehingga saat server mati catatan tetap aman di antrian offline dan terkirim pada percobaan berikutnya. Ini sesuai dengan aplikasi seluler yang sering kehilangan koneksi.
- Kelebihan: format JSON per baris mudah dibaca manusia dan mudah diperiksa dengan perintah dasar seperti type dan Get-Content.
- Keterbatasan: pipe di sini hanya menghubungkan proses berkerabat (induk dan anak) pada satu mesin dan dipakai satu arah; komunikasi dua arah atau antarproses tak berkerabat memerlukan mekanisme lain seperti named pipe atau socket.
- Keterbatasan: TCP tidak punya batas pesan, sehingga aplikasi wajib menangani batas pesan dan pembacaan berulang; satu kali recv() saja sudah merusak data pada volume besar (Tabel 6.2 no. 5).
- Keterbatasan: server melayani satu klien pada satu waktu, dan ACK hanya sekali per kiriman, bukan per catatan. Jika koneksi putus di tengah pengiriman, klien tidak tahu catatan mana yang sudah diterima, dan pengiriman ulang dapat menimbulkan duplikat; pada pengujian server_Lama.py sebagian catatan sudah tersimpan di server, padahal seluruhnya masuk antrian untuk dikirim ulang.

6.4 Rekomendasi

Berdasarkan keterbatasan di atas, kelompok merekomendasikan perbaikan berikut jika program dikembangkan menjadi sinkronisasi sungguhan:

- Beri tiap catatan identitas unik (misalnya gabungan id perangkat dan id catatan) dan buat server mengabaikan duplikat, agar pengiriman ulang aman.
- Gunakan batas pesan eksplisit, misalnya awalan panjang atau ACK per catatan, supaya server dan klien tahu persis catatan yang sudah diterima.
- Gunakan thread atau selectors pada server bila klien lebih dari satu.

- Tambahkan timeout dan percobaan ulang bertahap pada klien, serta enkripsi (TLS) bila data melewati jaringan sungguhan.

7. Kesimpulan

Kelompok berhasil mensimulasikan komunikasi antarkomponen aplikasi dengan dua mekanisme message passing, yaitu pipe antara proses Penyimpan dan Pengirim, dan socket TCP antara klien dan server. Program lulus pengujian pengiriman normal, server mati dengan antrian offline, dan pengiriman 1999 catatan. Dua masalah ditemukan dan diperbaiki: pembacaan satu kali recv(4096) yang memotong data besar, dan klien yang saling menunggu dengan server karena belum menutup sisi kirim. Pipe cocok untuk menyinkronkan proses berkerabat di satu perangkat, sedangkan socket menjangkau server tetapi menuntut aplikasi menangani batas pesan, duplikasi, dan kegagalan koneksi sendiri.

8. Kepingan Puzzle

Nomor kepingan	#4
Isi kepingan	Program IPC (client.py dan server.py), diagram alur komunikasi, tabel hasil dan pengujian, serta lembar kontribusi
Lokasi di portofolio	https://github.com/NazwaSR0/SO_k-1_kelompok-6/upload/main
Dipakai untuk	Level 2 Puzzle UTS : program, diagram, dan hasil uji ini menjadi artefak IPC (pipe dan socket) yang dirangkai bersama kepingan lain

Daftar Pustaka

Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2018). *Operating systems: Three easy pieces* (v1.00). Arpaci-Dusseau Books.

Kerrisk, M. (2010). *The Linux programming interface*. No Starch Press.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10th ed.). Wiley.

Lampiran A. Kode Program dan Log

Kode program lengkap. Server.py dan server_Lama.py sesuai berkas pada folder pengujian; client.py versi awal sesuai tangkapan layar editor.

server.py (server versi perbaikan, 1109 byte)

```
import socket, json

HOST, PORT = "127.0.0.1", 5000
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    s.bind((HOST, PORT))
    s.listen()
    print("Server menunggu klien...")
    while True:
        conn, addr = s.accept()
        with conn:
            chunks = []
            while True:
                part = conn.recv(4096)
                if not part:
                    break
                chunks.append(part)
            data = b"".join(chunks).decode()
            jumlah = 0
            for line in data.strip().split("\n"):
                if not line:
                    continue
                try:
                    rec = json.loads(line)
                except json.JSONDecodeError:
                    print("Baris rusak dilewati")
                    continue
                with open("server_db.jsonl", "a") as f:
                    f.write(json.dumps(rec) + "\n")
                jumlah += 1
            print("Diterima:", jumlah, "catatan")
            conn.sendall(b"ACK")
```

server_Lama.py (server versi awal, 639 byte)

```
import socket, json

HOST, PORT = "127.0.0.1", 5000
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.bind((HOST, PORT))
```

```
    s.listen()
    print("Server menunggu klien...")
    while True:
        conn, addr = s.accept()
        with conn:
            data = conn.recv(4096).decode()
            if not data:
                continue
            for line in data.strip().split("\n"):
                rec = json.loads(line)
                with open("server_db.jsonl", "a") as f:
                    f.write(json.dumps(rec) + "\n")
                print("Diterima:", rec)
            conn.sendall(b"ACK")
```

client.py (versi akhir, 1926 byte)

```
import socket, json, time, os
from multiprocessing import Process, Pipe

HOST, PORT = "127.0.0.1", 5000
ANTRIAN = "antrian_offline.jsonl"   # antrean offline (data tertahan saat server mati)

def muat_antrian():
    if not os.path.exists(ANTRIAN):
        return []
    with open(ANTRIAN) as f:
        return [json.loads(l) for l in f if l.strip()]

def simpan_antrian(data):
    with open(ANTRIAN, "w") as f:
        for x in data:
            f.write(json.dumps(x) + "\n")

# Proses 1: menghasilkan data, mengirim lewat PIPE
def penyimpan(conn):
    print("Penyimpan  pid =", os.getpid())
    for i in range(1, 6):
        conn.send({"id": i, "data": f"catatan-{i}", "waktu": time.time()})
    conn.send(None)  # tanda selesai
```

```
# Proses 2: menerima dari PIPE, mengirim ke server lewat SOCKET
def pengirim(conn):
    print("Pengirim   pid =", os.getpid())
    buffer = []
    while True:
        item = conn.recv()
        if item is None:
            break
        buffer.append(item)

    semua = muat_antrian() + buffer      # antrean lama + data baru
    try:
        with socket.create_connection((HOST, PORT), timeout=5) as s:
            s.sendall(("\n".join(json.dumps(x) for x in semua) + "\n").encode())
            s.shutdown(socket.SHUT_WR)   # beri tahu server: data selesai (EOF)
            balasan = s.recv(1024).decode()
            print("Balasan server:", balasan)
        if balasan == "ACK":
            if os.path.exists(ANTRIAN):
                os.remove(ANTRIAN)       # sukses -> antrean dihapus
    except OSError as e:
        simpan_antrian(semua)            # gagal -> simpan ke antrean offline
        print("Gagal kirim, data disimpan ke antrian offline:", e)

if __name__ == "__main__":
    print("Induk      pid =", os.getpid())
    a, b = Pipe()
    p1 = Process(target=penyimpan, args=(a,))
    p2 = Process(target=pengirim, args=(b,))
    p1.start(); p2.start()
    p1.join(); p2.join()
```

server.py

```
import socket, json

HOST, PORT = "127.0.0.1", 5000
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
```

```
s.bind((HOST, PORT))
s.listen()
print("Server menunggu klien...")
while True:
    conn, addr = s.accept()
    with conn:
        chunks = []
        while True:
            part = conn.recv(4096)
            if not part:
                break
            chunks.append(part)
        data = b"".join(chunks).decode()
        jumlah = 0
        for line in data.strip().split("\n"):
            if not line:
                continue
            try:
                rec = json.loads(line)
            except json.JSONDecodeError:
                print("Baris rusak dilewati")
                continue
            with open("server_db.jsonl", "a") as f:
                f.write(json.dumps(rec) + "\n")
            jumlah += 1
        print("Diterima:", jumlah, "catatan")
        conn.sendall(b"ACK")
```

Lampiran B. Lembar Kontribusi Anggota

Persentase kontribusi berjumlah 100% dan disepakati seluruh anggota.

No	Nama	NIM	Peran dan kontribusi	%	Paraf
1	Nazwa Salsabilla R	250660121077	Koordinasi kelompok, menyusun draf laporan, analisis sistem.	34%	

No	Nama	NIM	Peran dan kontribusi	%	Paraf
2	Satrio Pratama	250660121095	Pengembang program client.py, menangani socket & antrean, dan pengujian	33%	
3	Rizky Imam Andieka	250660121054	Pengembang program server.py, dokumentasi log.	33%	
	Total			100%	

Lampiran C. Log Kerja Kelompok

Tanggal	Kegiatan	Anggota hadir	Hasil
13 Oktober 2026	Menulis program awal (client.py dengan Pipe dan server_Lama.py), menguji pengiriman 5 catatan beberapa kali, dan memperbaiki IndentationError	Semua anggota	Server menerima dan menyimpan catatan ke server_db.jsonl; uji server mati menghasilkan WinError 10061
14 Oktober 2026	Memperbaiki server (membaca sampai koneksi ditutup), menambah antrian offline pada klien, menguji server mati, 1999 catatan, dan server lama, lalu menyusun laporan	Semua anggota	Seluruh skenario pada Tabel 6.2 terdokumentasi, laporan disusun

Lampiran D. Pernyataan Penggunaan AI Generatif

☐ Kelompok kami tidak menggunakan AI generatif dalam tugas ini.

☑ Menggunakan AI generatif dengan rincian berikut:

Alat	Tujuan penggunaan	Bagian yang dibantu	Cara memverifikasi
Claude	Membantu menyusun draf narasi laporan dan penyesuaian kode.	Pendahuluan	Dicocokkan oleh anggota kelompok dengan kode program eksekusi aktual.
ChatGPT	Mengoreksi langkahh langkah pengerjaan	5. Langkah Pengerjaan	Dicocokkan oleh anggota kelompok

Lampiran E. Daftar Periksa Sebelum Mengumpulkan

☑ Semua teks petunjuk abu-abu dan kotak petunjuk sudah dihapus.

☑ Identitas, kode tugas, dan nama host VM pada sampul sudah benar.

☑ Setiap poin deskripsi tugas sudah dijawab di bagian Analisis.

☑ Setiap tangkapan layar menampilkan prompt/nama host VM dan memiliki nomor serta keterangan.

☑ Kepingan puzzle sudah disimpan di portofolio dan tautannya dicantumkan.

☑ Lembar kontribusi sudah diisi dan disetujui semua anggota (total 100%).

☑ Kode dan konfigurasi sudah diunggah ke repositori kelompok.

☑ Pernyataan penggunaan AI generatif sudah diisi.

☑ Daftar isi sudah diperbarui (klik kanan → Update Field).

☑ Berkas disimpan sebagai PDF dengan nama <Kelas>_<Kode>_Kelompok<N>.pdf.

Swa-nilai berdasarkan kriteria lembar tugas

Kriteria dan bobot disalin dari lembar tugas K-1.

Kriteria (dari lembar tugas)	Bobot	Sudah terpenuhi? Di bagian mana?
Hasil/keberhasilan implementasi	35%	Ya. Bagian 5, Bagian 6.1 dan 6.2, serta Lampiran A
Kedalaman analisis	30%	Ya. Bagian 6.3 (tiga poin tugas) dan 6.4
Dokumentasi	20%	Ya. Bagian 4 dan 5 (gambar bernomor), diagram alur pada Bagian 5.2, serta Lampiran A dan C
Kerja sama dan pembagian peran	15%	Ya. Bagian 2 dan Lampiran B dan C